AMA-315330 Practical Exam

Same line-by-line performance-style codes + output + easy theory (Q1–Q16)

These codes are written with normal indentation and separate lines, like the programs shown in the chat. For questions with no exact graph in the exam sheet, an example is clearly labelled.

Q1. Implement Breadth First Search (BFS) Algorithm

Aim: Implement Breadth First Search (BFS) Algorithm.

Python Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []
queue = ['A']

while queue:
    node = queue.pop(0)

    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])
```

Expected Output

```
A B C D E F
```

Easy explanation: BFS visits nodes level by level using a queue. A queue follows FIFO (First In, First Out).

Remember: BFS = Queue + FIFO.

Q2. BFS for the Given Graph (Start Node 1)

Aim: BFS for the Given Graph (Start Node 1).

Python Code

```
graph = {
    1: [2, 4, 5],
    2: [3, 6, 7],
    3: [],
    4: [],
    5: [],
    6: [],
    7: []
}

visited = []
queue = [1]

while queue:
    node = queue.pop(0)

    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])
```

Expected Output

```
1 2 4 5 3 6 7
```

Easy explanation: BFS starts at node 1, visits its neighbours, and then visits the next level.

Remember: Follow the order in the graph's adjacency lists.

Q3. BFS Shortest Path for the Graph in the PDF

Aim: BFS Shortest Path for the Graph in the PDF.

Python Code

```
graph = {
    'A': ['B', 'E', 'C'],
    'B': ['A', 'E', 'D'],
    'E': ['A', 'B', 'D'],
    'D': ['B', 'E'],
    'C': ['A', 'F', 'G'],
    'F': ['C'],
    'G': ['C']
}

queue = [['A']]

while queue:
    path = queue.pop(0)
    node = path[-1]

    if node == 'G':
        print("Shortest Path:", path)
        break

    for n in graph[node]:
        if n not in path:
            queue.append(path + [n])
```

Expected Output

```
Shortest Path: ['A', 'C', 'G']
```

Easy explanation: BFS finds a path with the fewest edges in an unweighted graph. It stores paths in a queue.

Remember: Queue + Path + Destination. Note: the picture shows edge weights, but BFS does not use weights. The PDF does not explicitly state the start/goal; this example uses A to G.

Q4. Implement Depth First Search (DFS) Algorithm

Aim: Implement Depth First Search (DFS) Algorithm.

Python Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)

        for n in graph[node]:
            dfs(n)

dfs('A')
```

Expected Output

```
A B D E C F
```

Easy explanation: DFS explores one branch as far as possible before returning to another branch. This code uses recursion.

Remember: DFS = Recursion / Stack.

Q5. Implement BFS and DFS Algorithm

Aim: Implement BFS and DFS Algorithm.

Python Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': [],
    'E': []
}

# BFS
visited = []
queue = ['A']
```

```
while queue:
    node = queue.pop(0)
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])

print()

# DFS
visited = []

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for n in graph[node]:
            dfs(n)

dfs('A')
```

Expected Output

```
A B C D E
A B D C E
```

Easy explanation: BFS uses a queue and visits level by level. DFS uses recursion and explores a branch deeply.

Remember: BFS = Queue; DFS = Recursion.

Q6. Implement DFS Using Stack

Aim: Implement DFS Using Stack.

Python Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': [],
    'E': []
}

stack = ['A']
visited = []

while stack:
    node = stack.pop()

    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        stack.extend(reversed(graph[node]))
```

Expected Output

```
A B D C E
```

Easy explanation: A stack follows LIFO (Last In, First Out). DFS pops a node, visits it, and pushes its neighbours.

Remember: DFS stack = LIFO.

Q7. DFS Traversal for the Given Graph (Start A)

Aim: DFS Traversal for the Given Graph (Start A).

Python Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C']
}

visited = []

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for n in graph[node]:
            dfs(n)
```

```
dfs('A')
```

Expected Output

```
A B D E C F
```

Easy explanation: DFS visits a node and then recursively visits its unvisited neighbours. The visited list prevents loops.

Remember: Check visited before recursive call.

Q8. Implement A* Search (Informed Search)

Aim: Implement A* Search (Informed Search).

Python Code

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('G', 4)],
    'C': [('G', 2)],
    'G': []
}
h = {'A': 4, 'B': 3, 'C': 2, 'G': 0}

open_list = [(h['A'], 0, 'A', ['A'])]

while open_list:
    open_list.sort()
    f, cost, node, path = open_list.pop(0)

    if node == 'G':
        print("Path:", path, "Cost:", cost)
        break

    for nxt, weight in graph[node]:
        new_cost = cost + weight
        open_list.append((new_cost + h[nxt], new_cost,
                          nxt, path + [nxt]))
```

Expected Output

```
Path: ['A', 'C', 'G'] Cost: 5
```

Easy explanation: A* is an informed search algorithm. It uses f(n) = g(n) + h(n), where g is the cost so far and h is the estimated cost to the goal.

Remember: A* = g + h. The exam sheet does not clearly provide a graph/heuristic table for Q8; this is an example.

Q9. Create and Display a Decision Tree

Aim: Create and Display a Decision Tree.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

data = load_iris()

model = DecisionTreeClassifier(max_depth=3, random_state=0)
model.fit(data.data, data.target)

plot_tree(model,
          feature_names=data.feature_names,
          class_names=data.target_names,
          filled=True)
plt.show()
```

Expected Output

```
A decision-tree diagram is displayed.
```

Easy explanation: A decision tree predicts a class using feature-based questions. Internal nodes are tests and leaves are predictions.

Remember: fit() trains the model; plot_tree() displays it.

Q10. Implement Bayes' Theorem

Aim: Implement Bayes' Theorem.

Python Code

```
p_a = 0.4
p_b_given_a = 0.8
p_b = 0.5
```

```
p_a_given_b = (p_b_given_a * p_a) / p_b

print(round(p_a_given_b, 2))
```

Expected Output

```
0.64
```

Easy explanation: Bayes' theorem: P(A|B) = P(B|A) × P(A) / P(B).

Remember: Multiply, then divide.

Q11. Confusion Matrix and Model Measures

Aim: Confusion Matrix and Model Measures.

Python Code

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score

actual = [1, 0, 1, 1, 0, 1]
predicted = [1, 0, 1, 0, 0, 1]

print(confusion_matrix(actual, predicted))
print("Accuracy:", accuracy_score(actual, predicted))
print("Precision:", precision_score(actual, predicted))
print("Recall:", recall_score(actual, predicted))
```

Expected Output

```
[[2 0]
 [1 3]]
Accuracy: 0.8333...
Precision: 1.0
Recall: 0.75
```

Easy explanation: A confusion matrix compares actual and predicted labels. Accuracy = correct/total; Precision = TP/(TP+FP); Recall = TP/(TP+FN).

Remember: Accuracy = overall correct; Precision = predicted positives correct; Recall = actual positives found.

Q12. Simple Linear Regression

Aim: Simple Linear Regression.

Python Code

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4]]
y = [2, 4, 6, 8]

model = LinearRegression()
model.fit(X, y)

print(model.predict([[5]])[0])
```

Expected Output

```
10.0
```

Easy explanation: Simple linear regression learns a line y = mx + c to predict a numerical value from one input.

Remember: fit() learns the line; predict() estimates a value.

Q13. Find Shortest Path Using A* Algorithm

Aim: Find Shortest Path Using A* Algorithm.

Python Code

```
# Example graph; use the teacher's graph if supplied
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2)],
    'C': [('D', 1)],
    'D': []
}
h = {'A': 3, 'B': 2, 'C': 1, 'D': 0}

open_list = [(h['A'], 0, 'A', ['A'])]

while open_list:
    open_list.sort()
    f, cost, node, path = open_list.pop(0)
```

```
    if node == 'D':
        print("Path:", path, "Cost:", cost)
        break

    for nxt, weight in graph[node]:
        new_cost = cost + weight
        open_list.append((new_cost + h[nxt], new_cost,
                          nxt, path + [nxt]))
```

Expected Output

```
Path: ['A', 'B', 'D'] Cost: 3
```

Easy explanation: A* uses f(n) = g(n) + h(n) to select a promising path with a low estimated total cost.

Remember: Remember f = g + h. This is an example because the sheet image does not show the required graph.

Q14. Calculate Cross-Validation Score (Iris)

Aim: Calculate Cross-Validation Score (Iris).

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

data = load_iris()
model = DecisionTreeClassifier(random_state=0)

scores = cross_val_score(model, data.data, data.target, cv=5)

print(scores)
print("Mean:", scores.mean())
```

Expected Output

```
Prints five scores and their mean. Exact values can vary by library version.
```

Easy explanation: Cross-validation divides data into folds and evaluates the model repeatedly using different folds.

Remember: cv=5 means five folds.

Q15. Split Dataset into Training and Testing Sets

Aim: Split Dataset into Training and Testing Sets.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=0
)

print(X_train.shape)
print(X_test.shape)
```

Expected Output

```
(120, 4)
(30, 4)
```

Easy explanation: Training data teaches the model. Testing data checks the model on unseen data.

Remember: 80% training + 20% testing.

Q16. BFS for the Given Graph (Start Node 5)

Aim: BFS for the Given Graph (Start Node 5).

Python Code

```
graph = {
    5: [3, 7],
    3: [2, 4],
    7: [8],
    2: [],
    4: [8],
    8: []
}

visited = []
queue = [5]
```

```
while queue:
    node = queue.pop(0)

    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])
```

Expected Output

```
5 3 7 2 4 8
```

Easy explanation: BFS visits nodes level by level from node 5. The queue follows FIFO order.

Remember: BFS = Queue + FIFO.